Statement of Purpose

Evan Williams *Carnegie Mellon University*

1 Introduction

My research interests lie in hardware/software co-design, specifically compilers, computer architecture, and programming languages. As computer system performance plateaus, engineers increasingly rely on specialized hardware, but specialization degrades the hardware/software interface, making it challenging to fully leverage accelerators. I'm interested in program representations that preserve high-level structure throughout lowering, since this structure determines how we schedule and optimize computations across different hardware architectures.

My goal is to democratize specialized computing by developing tools and abstractions for hardware that support more domains. As a research assistant, I worked with Calyx, an intermediate representation that targets hardware designs, to create hardware accelerators for computational genomics and machine learning. I aim to build on this work by rethinking hardware/software abstractions to improve performance and productivity. **I am fortunate to be supported by an NSF graduate fellowship that provides three years of funding for my research**.

2 Capra Research Group at Cornell University

2.1 Accelerated Pangenome Graph Queries

Pangenome graphs provide a complete representation of the mutual alignment of genome collections. However, analyzing hundreds of gigabase-scale genomes using pangenome graphs is computationally expensive. The goal of this project was to design a domain-specific language (DSL) to generate custom hardware accelerators for pangenomic graph queries using Calyx.

I used Dahlia, a programming language for generating FPGA designs that compiles to Calyx, to implement these queries. Although our long-term goal was a custom DSL, Dahlia provided a practical starting point for targeting Calyx. I referenced `odgi`, a C++ library with the fastest available software implementations. The main challenge was that `odgi` relied on performance optimizations that could not be directly expressed in Dahlia. Nonetheless, I implemented a fixed-function version of the *node depth* query and presented the results to our collaborators, who provided guidance on further improvements. This work showed improvements on fixed-size inputs and highlighted how essential domain-specific context is when designing accelerators, reinforcing my interest in compiler design for cross-domain hardware.

2.2 Compiling PyTorch Models to Hardware Accelerators

For my master's project, I explored compilation techniques for lowering PyTorch models to FPGA-executable hardware using Calyx. Mapping directly from PyTorch to hardware is challenging because the semantics of high-level tensor programs are lost when lowered to static hardware descriptions. By targeting Calyx, we could preserve structured control and dataflow information, allowing us to better reason about parallelism and memory reuse during compilation.

My work lowered a feedforward neural network with one hidden layer and a ReLU activation function from PyTorch to Calyx. Using Allo, an accelerator design language, I lowered PyTorch to MLIR, optimized the design with the `affine` and `scf` dialects, and then emitted Calyx using CIRCT. One challenge arose when handling weights through the compilation path. In MLIR, weights are represented as multi-dimensional `memref` values embedded in the program, while Calyx treats memories as explicit hardware components with external initialization, causing weight data to be lost during CIRCT's lowering to Calyx. The core insight was that weights needed to be materialized as explicit Calyx memories rather than carried as MLIR values. I implemented a CIRCT pass that flattens `memref` tensors, emits the corresponding Calyx `@external`

memory declarations, and generates JSON initialization files. I also extended Calyx to support floating-point operations using HardFloat.

I ran the optimized hardware design on an FPGA using Vivado HLS and AXI interfaces, completing the full HLS path. In our evaluation, the Calyx-generated accelerator achieved a 99× speedup over the software baseline and was only 1.5× slower than a design produced by Vitis HLS, a closed-source industrial tool. Future research could explore support for more models and optimizations for backpropagation. **This work is being submitted to the Compilers for Machine Learning (C4ML) workshop at CGO 2026**.

3 Professional Experience at Amazon Web Services (AWS)

I currently work as a software engineer at AWS on the HealthOmics team. HealthOmics executes workflows written in WDL, Nextflow, and CWL and orchestrates the computation on AWS. I recently implemented lazy loading in the WDL engine, which delays downloads of intermediate task outputs from the caching layer until they are needed. This required reasoning about the trade-off between a single bulk download and multiple smaller downloads. For some workflows, a single large download is faster, while for others, lazy loading performs better. I designed a static analysis pass over the workflow's AST to choose the appropriate strategy, reducing tail latency for some workloads by 11% and lowering AWS costs.

Working on HealthOmics has accentuated the compiler-like challenges pervasive in industry. Each workflow describes a computation graph that must be analyzed, scheduled, and executed. However, current workflow representations collapse execution and scheduling decisions and provide limited semantic structure for optimization. As a result, the system cannot reason about cost models, data locality, or hardware specialization. These limitations drive my interest in designing compiler infrastructures that unlock more aggressive optimizations on heterogeneous computing systems.

4 Conclusion

Looking ahead, my research goals are guided by two key questions: (1) How can we make accelerators easier to program? (2) How can we make accelerators easier to develop? More broadly, I am interested in how compilers and programming models can expose structure in computations, reason about parallelism and memory, and simplify cross-domain problems.

CMU offers one of the strongest environments to pursue these goals with several faculty at the intersection of compilers, computer architecture, and programming languages. I am especially interested in working with **Professor Olivia Hsu**, **Professor Brandon Lucia**, and **Professor Nathan Beckmann**. Professor Hsu's work on sparse tensor representations closely aligns with my interest in rethinking hardware/software boundaries for accelerators. I'm excited to contribute to her broader vision of bringing domain-specific accelerator performance to all applications. Professor Lucia's contributions to programming for spatial dataflow architectures resonate with my desire to build systems that preserve semantics throughout the stack. Professor Beckmann's research on polymorphic cache hierarchies for optimizing data movement epitomizes the approach to co-design I'm eager to explore.

I am drawn to CMU's focus on research with real-world impact and to its collaborative culture, where students are encouraged to engage deeply with research from the beginning. After completing my PhD, I hope to lead an academic or industrial research group advancing hardware/software co-design and mentoring the next generation of researchers.